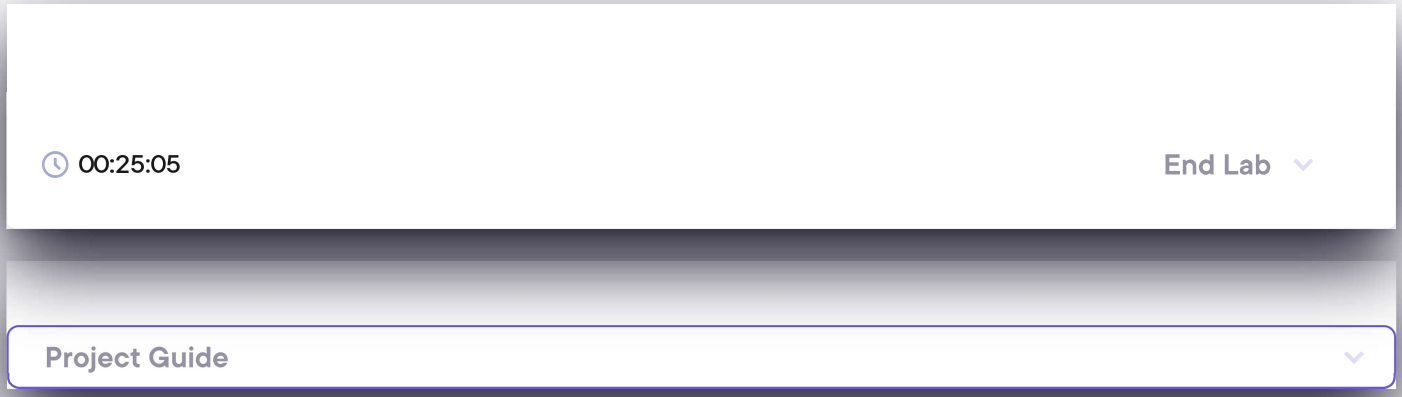
Common Operations on a PostgreSQL Database

By A Cloud Guru



Common Operations on a PostgreSQL Database

Introduction

In this lab we perform some common operations on a database. We create a database, add a table, and fill the table from a `csv` file.

Then we update the database table with a new record, change a record, and finally read form the database table to make sure these operations succeeded.

Scenario

You are a freelance developer who has accepted a job to develop a database for a veterinarian company. They have given you a small `CSV` file and asked you to populate the database with a single table holding the data in the file. There are notes attached. One asks you to add a weight for Petra of 12.5 lbs. The other asks you to add a new pet Esmerelda who is a 2.5 yr old female Angus cow that weighs 1250 lbs, has no health issues, is vaccinated, and is owned by the GarciaRanch.

Connecting to the Lab

1. Begin by logging in to the lab server using the credentials provided on the hands-on lab page.

```
ssh cloud_user@PUBLIC_IP_ADDRESS
```

Start Jupyter Notebook Server and Access on the Local Machine

1. Activate the conda environment.

   ```
   conda activate base
   ```

2. Transfer directories.

   ```
   cd hol
   ls
   ```

3. Get the notebook token.

   ```
   python get_notebook_token.py
   ```

4. Copy the notebook token displayed in the terminal. The will be referred to as NOTEBOOK_TOKEN throughout the rest of this lab.

5. On the local machine, open a terminal and enter the following command. Make sure to use the public IP address from the lab credentials page. When asked for a password, use the password on the lab credentials page.

   ```
   ssh -N -L localhost:8087:localhost:8086
   cloud_user@PUBLIC_IP_ADDRESS
   ```

6. Leave the terminal open.

7. Open a browser and navigate to http://localhost:8087 which will open a connection to Jupyter.

8. Paste NOTEBOOK_TOKEN into the box provided and click **Log in**.

Create the Database and Import Packages Needed

1. Navigate back to the lab server terminal (the first terminal used and where we obtained the token).

2. Enter the postgres command line using the lab credentials on the lab page.

   ```
   sudo -u postgres psql
   ```

3. Create the database.

   ```
   CREATE DATABASE cloud_user;
   ```

4. Create the user.

   ```
   CREATE USER cloud_user WITH ENCRYPTED PASSWORD 'cloud_user';
   ```

5. Grant the newly-created user all access to the newly-created database.

```
GRANT ALL PRIVILEGES ON DATABASE cloud_user TO cloud_user;
```

6. Quit the command line.

```
\q
```

7. Install necessary packages. This is done later in the lab, but placed here for readability and to avoid a later error.

```
conda install psycopg2
```

8. Navigate back to the browser view of the Jupyter notebook and open the **lab** folder. Then open the `hol_3.1.1_solution` notebook. This step is not in the video.

9. Evaluate the following cell.

```
import pandas as pd
import psycopg2

CONNECT_DB = "host=localhost port=5432 dbname=cloud_user
user=cloud_user password=cloud_user"
```

Create a `customers` Table and Fill It with the Data in the `vets.csv` File

1. In the Jupyter notebook, evaluate the following cell to create the table.

```
create_table_query = '''CREATE TABLE customers (
    id SERIAL PRIMARY KEY,
    name varchar (25),
    owner varchar (25),
    type varchar (25),
    breed varchar (25),
    color varchar (25),
    age smallint,
    weight float4,
    gender varchar (1),
    health_issues boolean,
    indoor_outdoor varchar(10),
    vaccinated boolean
); '''

try:
    # Make connection to db
    cxn = psycopg2.connect(CONNECT_DB)
```

```
        # Create a cursor to db
        cur = cxn.cursor()

        # Send sql query to request
        cur.execute(create_table_query)
        records = cxn.commit()

    except (Exception, psycopg2.Error) as error :
        print ("Error while connecting to PostgreSQL", error)

    finally:
        #closing database connection.
        if(cxn):
            cur.close()
            cxn.close()
            print("PostgreSQL connection is closed")

        print(f'Records: {records}')
```

2. Evaluate the following cell to populate the table.

```
try:
    # Make connection to db
    cxn = psycopg2.connect(CONNECT_DB)

    # Create a cursor to db
    cur = cxn.cursor()

    # read file, copy to db
    with open('./vet.csv', 'r') as f:
        # skip first row, header row
        next(f)
        cur.copy_from(f, 'customers', sep=",")
        cxn.commit()

except (Exception, psycopg2.Error) as error :
    print ("Error while connecting to PostgreSQL", error)

finally:
    #closing database connection.
    if(cxn):
        cur.close()
```

```
        cxn.close()
        print("PostgreSQL connection is closed")
        print("customers table populated")
```

Create a Function to Fetch Data from the Database and Test It

1. Evaluate the following cell to create a method `db_server_fetch`.

```
def db_server_fetch(sql_query):
    try:
        # Make connection to db
        cxn = psycopg2.connect(CONNECT_DB)

        # Create a cursor to db
        cur = cxn.cursor()

        # Send sql query to request
        cur.execute(sql_query)
        records = cur.fetchall()

    except (Exception, psycopg2.Error) as error :
        print ("Error while connecting to PostgreSQL", error)

    finally:
        #closing database connection.
        if(cxn):
            cur.close()
            cxn.close()
            print("PostgreSQL connection is closed")
        return records
```

2. Test the function by evaluating the following cell.

```
select_query = '''SELECT * FROM customers;'''

records = db_server_fetch(select_query)
print(records)
```

Create a Function to Update the Database and Make the Requested Changes

1. Evaluate the following cell to create a method `db_server_change`.

```
def db_server_change(sql_query):
    try:
        # Make connection to db
        cxn = psycopg2.connect(CONNECT_DB)

        # Create a cursor to db
        cur = cxn.cursor()

        # Send sql query to request
        cur.execute(sql_query)
        records = cxn.commit()

    except (Exception, psycopg2.Error) as error :
        print ("Error while connecting to PostgreSQL", error)

    finally:
        #closing database connection.
        if(cxn):
            cur.close()
            cxn.close()
            print("PostgreSQL connection is closed")
        return records
```

2. Evaluate the following cell to add the `Esmerelda` account.

```
add_data = '''INSERT INTO customers
    (id, name, owner, type, breed, color, age, weight, gender,
health_issues, indoor_outdoor, vaccinated)
    VALUES
    (7, 'Esmerelda', 'Garcia Ranch', 'Cattle', 'Angus', 'black',
2.5, 1250, 'f', false, 'outdoor', true);'''

db_server_change(add_data)
```

3. Evaulate the following cell to verify the record is updated.

```
select_query = '''SELECT * FROM customers WHERE name =
'Esmerelda';'''
```

```
records = db_server_fetch(select_query)
print(records)
```

4. Evaluate the following cell to update Petra's weight.

```
update_data = '''UPDATE customers SET weight = 12.5 WHERE name =
'Petra';'''

db_server_change(update_data)
```

5. Evaluate the following cell to verify the changes.

```
select_query = '''SELECT * FROM customers WHERE name = 'Petra';'''

records = db_server_fetch(select_query)
print(records)
```

Conclusion

Congratulations, you've completed this hands-on lab!

Lab Tools

Lab Diagram | Instant Terminal

Lab Credentials

Help ↗

It is recommended that all Hands-on Labs are opened in an **incognito window**.

Cloud Server Workstation

Username

cloud_user

Password

#UloHzd8

Private IP address of Workstation

10.0.1.187

Public IP address of Workstation

54.146.217.64

Learning Objectives

Successfully complete this lab by achieving the following learning objectives.

1. Start Jupyter Notebook Server and Access on the Local Machine
2. Create the Database and Import Packages Needed
3. Create a `customers` Table in the Database and Fill It with the Data in the `vets.csv` File
4. Create a Function to Fetch Data from the Database and Test It
5. Create a Function to Update the Database and Make the Requested Changes